Chapter 2.1: A Low-Dimensional Subspace

Overview

1. Principal Component Analysis (PCA)
2. Power Iteration
3. Probabilistic PCA
4. Matrix Completion

Principal Component Analysis

Motivation

- Low-Rank Assumption: Our data, which has D dimensions, can be reduced to a lower dimensional subspace, d.
- Represent subspace $S \subseteq \mathbb{R}^D$ of dimension d by orthonormal matrix $U \in O(D, d) \subseteq \mathbb{R}^{D \times d}$ such that the column of U span S. Then, our data $\{x_i\}_{i=1}^N \subseteq \mathbb{R}^D$ have approximate low rank structure if

$$\begin{aligned} x_i &= U z_i + \epsilon_i, \forall i \in [N], \\ \{z_i\}_{i=1}^N &\subseteq \mathbb{R}^d, \\ \{\epsilon_i\}_{i=1}^N &\subseteq \mathbb{R}^D \end{aligned}$$

- Goal is to recover S

How to recover S

- Find U^*, z_i^* that best approximates $x_i \approx U^* z_i^*$.

$$\min_{\tilde{U}, \{\tilde{z}_i\}_{i=1}^N} \frac{1}{N} \sum_{i=1}^{N} ||x_i - \tilde{U}\tilde{z}_i||_2^2$$

- Simplify the problem (proof as exercise)

$$= \frac{1}{N} \sum_{i=1}^{N} \min_{\tilde{z}_i} ||x_i - \tilde{U}\tilde{z}_i||_2^2$$

$$= \frac{1}{N} \sum_{i=1}^{N} ||x_i - \tilde{U}\tilde{U}^T x_i||_2^2$$

$$\implies \min_{\tilde{U}, \{\tilde{z}_i\}_{i=1}^N} \frac{1}{N} \sum_{i=1}^{N} ||x_i - \tilde{U}\tilde{z}_i||_2^2 = \min_{\tilde{U}} \frac{1}{N} \sum_{i=1}^{N} ||x_i - \tilde{U}\tilde{U}^T x_i||_2^2$$

- This changes the problem to a "denoising problem." Goal becomes to approximate "denoiser", $U^*(U^*)^T \approx UU^T$, that projects x_i onto the subspace S

Computing Subspace Basis

- Let $X = [x_1, .., x_N] \in \mathbb{R}^{D \times N}$
- (IMPORTANT: for this notation, each *column* is a data point, NOT each row)

$$\arg\min_{\tilde{U}} \frac{1}{N} \sum_{i=1}^{N} ||x_i - \tilde{U}\tilde{U}^T x_i||_2^2 = \arg\max_{\tilde{U}} \frac{1}{N} \sum_{i=1}^{N} ||\tilde{U}^T x_i||_F^2$$

$$= \arg\max_{\tilde{U}} tr\{\tilde{U}^T (\frac{XX^T}{N}) \tilde{U}\}$$

- After proof by induction (base case proven in book, rest of proof left as exercise)...

Theorem 2.1

- If our data takes the form

$$x_i = U z_i + \epsilon_i,$$

then the principal components are the top d eigenvectors of XX^T/N.

Remarks

- If oriented in such a way where *rows* are data points (i.e. $X \in \mathbb{R}^{N \times D}$), then its the top d eigenvectors of $X^T X / N$.
- May not know true dimension of subspace. Evaluate using either scree plot or through "hyper parameter optimization," usually by brute force or Bayesian optimization.
- Neural Network Interpretation: can be viewed as a *simple two-layer weight-tied linear neural network* (simple??? it has eight words??????)

$$\underbrace{\underbrace{\underbrace{\text{denoise}(\boldsymbol{x}) = \boldsymbol{U}^\star \circ}_{\text{output of "NN"}} \underbrace{\underbrace{\text{id} \circ \underbrace{(\boldsymbol{U}^\star)^\top \boldsymbol{x}}_{\text{first "layer"}}}_{\text{post-activation of first "layer"}}}_{\text{output of "NN"}}}$$

Power Iteration

Theorem 2.2

Let λ_i be ith eigenvalue of M and w_i be ith eigenvector of M. Assume eigenvalues $\lambda_1 > \lambda_i$. If we compute the fixed point

$$w = \frac{Mw}{||Mw||_2},$$

using the following iteration:

$$v_0 \sim N(0, 1),$$
$$v_{t+1} \leftarrow \frac{Mv_t}{||Mv_t||_2}$$

then, in the limit, v_t will converge to a top unit-norm eigenvector of M.

Algorithm

1. Apply Theorem 2.2 to find top eigenvector $v_t = w_1$ of M.
2. λ_1 is estimated by $w_1^T M w_1$
3. Apply Theorem 2.2 to $M - \lambda_1 v_1 v_1^T$ to find w_2 and λ_2.

- This can repeated until top d eigenvectors are found.

Probabilistic PCA

How Does It Compare to PCA?

- Same setup, but x_i and z_i are now random variables.

$$x = Uz + \epsilon$$

- Now, we solve problem with expectation:

$$\min_{\tilde{U},\tilde{z}} \mathbb{E}||x - \tilde{U}\tilde{z}||_2^2$$

$$\implies \arg\max_{\tilde{U}} tr\{\tilde{U}^T \mathbb{E}[xx^T]\tilde{U}\}$$

Solution and Remarks

- Solution is now top d eigenvectors of $\mathbb{E}[xx^T]$
- Previous formulation, XX^T/N, is actually an estimate of $\mathbb{E}[xx^T]$
- If we further assume $\mathbb{E}[z] = 0$ and $\mathbb{E}[\epsilon] = 0$, then $\mathbb{E}[x] = U\mathbb{E}[z] + \mathbb{E}[\epsilon] = 0$ and therefore $Cov(x) = \mathbb{E}[xx^T]$.
- Side note: One technique I've encountered is subtracting their data matrix X by the row means. This allows for $\mathbb{E}[x'] \approx 0$ and therefore you can find the eigenvectors of $Cov(x')$.

Matrix Completion

Setup

- Let data be $X = [x_1, .., x_n]^T \in \mathbb{R}^{D \times N}$. Our data gets corrupted from mask $M \in \{0, 1\}^{D \times N}$ known by us,

$$Y = M \odot X$$

Most Exteme Cases

1. Each entry that was masked in X was chosen independently, then there is no hope in recovering X exactly
2. X has rank 1, and mask is

$$M = \begin{pmatrix} 1_{(D-1)\times 1} & 0_{(D-1)\times(N-1)} \\ 1 & 1_{1\times(N-1)} \end{pmatrix},$$

   then we can reconstruct X with perfect accuracy (because we know one full data point and at least one value of each data point).

- Most problems are in between these two extremes
- *The lower-dimensional and more structured the data distribtuion is, the easier it is to process, and the fewer observations are needed - provided that the algorithm effectively utilizes this low-dimensional structure.*